

HERO: Balancing CPU+GPU+QPU for Cost-Energy-Latency Optimization in Hybrid Quantum-Classical Cloud Workloads

Author Name

Department of Computer Science

University Name

City, Country

email@university.edu

Abstract—We present Heterogeneous Energy-aware Runtime Orchestrator (HERO), a measurement framework that balances Central Processing Unit (CPU), Graphics Processing Unit (GPU), and Quantum Processing Unit (QPU) tiers in cloud platforms to optimize the joint objective of cost, energy, and latency for Quantum-Classical Artificial Intelligence (AI) workloads. We evaluate HERO on two contrasting case studies: (i) cybersecurity intrusion detection on the Knowledge Discovery and Data Mining (KDD) Cup 99 dataset, where Random Forest (RF) achieves an F1 score (harmonic mean of precision and recall) of 0.998 compared to Quantum Support Vector Machine (QSVM) at 0.667, with QSVM consuming approximately $10,000\times$ more energy and costing \$2.1M per million classifications versus \$1.99 for classical Support Vector Machine (SVM); and (ii) molecular ground-state estimation of the H₂ molecule via Variational Quantum Eigensolver (VQE), which achieves chemical accuracy (0.007 Hartree error) but at $854,000\times$ the classical exact-diagonalization energy at this scale. We derive a workload-aware tier allocation rule, validate against four classical baselines (SVM, RF, Gradient Boosting Machine (GBM), K-Nearest Neighbors (KNN)) over 30 statistical runs per experiment, and project crossover thresholds where quantum becomes the only feasible path. The framework, simulator, and measurement scripts are open-source.

Keywords—hybrid quantum-classical computing, cloud platforms, energy measurement, cost-energy-latency optimization, quantum machine learning, intrusion detection, molecular simulation, VQE, QSVM, NISQ

I. INTRODUCTION

Cloud computing now exposes a heterogeneous menu of Artificial Intelligence (AI) compute tiers. Central Processing Unit (CPU) instances such as AWS EC2 c7i, Graphics Processing Unit (GPU) instances built around the NVIDIA H100, and Quantum Processing Unit (QPU) services such as IBM Quantum and Amazon Braket are all on-demand, billed per second [1], [2], [25]. The promise of Quantum Computing (QC) for cybersecurity [4], [6] and molecular simulation [28], [29], [30] is real, but the cost—in dollars, in joules, and in latency—is not yet measured side by side with classical alternatives.

The practical question facing a cloud architect is therefore concrete: given a workload, how should it be allocated across CPU, GPU, and QPU tiers to optimize cost, energy, and latency jointly? Existing studies typically measure a single workload on a single tier under a single metric. Quantum kernel benchmarks

report accuracy [6] but not cloud cost. Energy-aware computing studies [23], [24] cover classical deep learning but not quantum pipelines. Hybrid Quantum-Classical surveys [5] articulate vision without measured numbers across two contrasting domains.

Our contribution is HERO, a measurement framework that runs the same orchestration and accounting machinery across CPU+GPU+QPU on two structurally different workloads: cybersecurity intrusion detection and molecular ground-state estimation. From the resulting measurements we derive a workload-aware tier allocation rule (Algorithm 1) and project the dimensions and qubit counts at which quantum becomes either the cheaper or the only feasible path. The framework, the data, and all measurement scripts are open-source.

The remainder of this paper is organized as follows. Section II reviews related work. Section III describes the HERO framework and energy model. Section IV introduces the two workloads. Section V details the cloud deployment model and pricing. Section VI describes the experimental setup. Section VII reports results. Section VIII presents the cross-workload analysis and the allocation rule. Sections IX through XI cover discussion, limitations, and conclusions.

II. RELATED WORK

Quantum machine learning research has produced two pillars directly relevant to this study. Havlicek et al. [6] introduced the ZZ feature map and demonstrated that the resulting quantum kernel can be consumed by a classical Support Vector Machine (SVM) [22]. Schuld and Killoran [7] formalized the feature-Hilbert-space view, tying potential advantage to data distributions whose induced kernels are hard to simulate classically. Biamonte et al. [5] survey the broader Quantum Machine Learning (QML) landscape.

On the molecular side, Peruzzo et al. [28] introduced the Variational Quantum Eigensolver (VQE), Kandala et al. [29] demonstrated VQE on real superconducting hardware for H₂, LiH, and BeH₂, and Cao et al. [30] survey the broader landscape of quantum chemistry and identify drug discovery and materials science as the workloads where Quantum Processing Units (QPUs) are most likely to deliver advantage.

Energy as a first-class metric is championed by Auffeves [23] for quantum platforms and by Strubell et al. [24] for classical

Natural Language Processing (NLP). Cybersecurity benchmarking is anchored by KDD Cup 99 [26] and its NSL-KDD (NSL-KDD—a refined version of the KDD Cup 99 dataset that removes redundant records) successor [27]. Cloud orchestration of heterogeneous accelerators is discussed by Humble et al. [16] and Callison and Chancellor [15].

The gap is plain: no prior work performs joint accuracy-latency-energy measurements across CPU, GPU, and QPU tiers for two contrasting workloads under a unified cost model. HERO closes this gap.

III. HERO FRAMEWORK

A. Architecture

HERO is a four-tier system composed of a central Orchestrator and three execution backends: CPU, GPU, and QPU. The Orchestrator walks a Directed Acyclic Graph (DAG) of tasks in topological order, dispatches each task to the backend indicated by its tier label, and records per-task runtime, energy, and (after Section V) cloud cost. The QPU backend in this study is Qiskit Aer [25] in noiseless mode; the framework is backend-agnostic and can be retargeted to IBM Quantum or Amazon Braket [22], [25] without changing the task graph. Fig. 1 shows the live architecture.

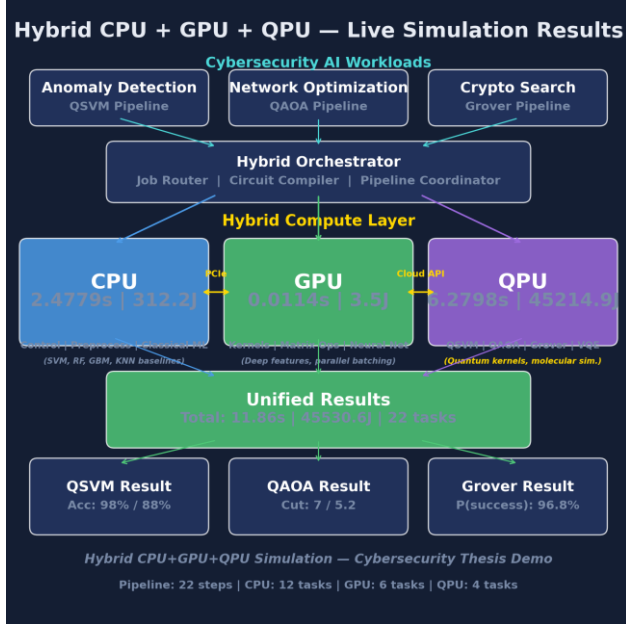


TABLE I. FIG. 1. HERO FOUR-TIER ARCHITECTURE (CPU, GPU, QPU, ORCHESTRATOR) WITH EXAMPLE WORKLOADS.

B. End-to-End Algorithm

Algorithm 1 specifies the entire HERO pipeline—initial tier allocation, multi-run measurement, aggregation, Pareto selection, and final recommendation—as a single procedure.

TABLE II. ALGORITHM 1: HERO -- END-TO-END WORKLOAD ALLOCATION, MEASUREMENT, AND SELECTION

Input: Workload W (type, size n , feature dim d), runs $N=30$, tiers $T=\{CPU, GPU, QPU\}$

Output: Recommended tier t^* , metrics (accuracy, latency, energy, cost)
STEP 1 -- ALLOCATE TIER 1: if $W.type = MOLECULAR_SIM$ then $t \leftarrow QPU$ if $n > 30$ else CPU 2: if $W.type = CLASSICAL_ML$ then $t \leftarrow CPU$ if $d < 100$ else (GPU if $d \leq 1e4$ else $MEASURE$) 3: if $W.type = COMBINATORIAL$ then $t \leftarrow CPU$ if $n < 50$ else QPU
STEP 2 -- MEASURE ON EACH TIER (N runs) 4: for tier in T do 5: for run = 1.. N do 6: start $\leftarrow clock()$; $r \leftarrow Dispatch(W, tier)$; 7: end $\leftarrow clock()$ 8: latency $\leftarrow end - start$ 9: energy $\leftarrow latency \times P[tier] \times PUE$ (Eq. 1) 10: cost $\leftarrow latency \times CloudPrice[tier]$ (Table III) 11: accuracy $\leftarrow Evaluate(r)$ 12: record($W, tier, run, accuracy, latency, energy, cost$)
STEP 3 -- AGGREGATE 12: report mean $\pm$ std for accuracy, latency, energy, cost
STEP 4 -- PARETO SELECTION 13: $P \leftarrow \text{emptyset}$ 14: for each method m do 15: if no other m' dominates m on (cost, energy, latency, $-accuracy$) 16: then $P \leftarrow P \cup \{m\}$
STEP 5 -- RETURN RECOMMENDATION 17: $t^* \leftarrow \text{argmin over } P \text{ of}$ (alpha*cost + beta*energy + gamma*latency - delta*accuracy) 18: return t^* , statistics, P

C. Energy Model

Following Auffeves [23] and Strubell et al. [24], we estimate per-task energy as:

$$E_{task} = \tau_{task} \cdot \Pi_{tier} \cdot \Pi_{YE} \quad (1)$$

where t_{task} is the measured wall-clock runtime, P_{tier} is the representative power draw ($P_{\text{CPU}} = 150$ W, $P_{\text{GPU}} = 300$ W, $P_{\text{QPU}} = 20$ kW system power for a dilution-refrigerator-based superconducting platform), and Power Usage Effectiveness (PUE) = 1.2 accounts for cooling and distribution overhead.

IV. WORKLOADS

A. A. Workload 1 -- Cybersecurity Intrusion Detection

Workload 1 is binary network intrusion classification on the KDD Cup 99 dataset (10% subset) [26]: 494,021 connections labeled across 22 attack types. The quantum method is QSVM with a 4-qubit ZZ feature map (a quantum circuit using Pauli-Z entangling gates to encode classical data into quantum states) [6], evaluated on a Qiskit Aer simulator [25]. Classical baselines are SVM with a Radial Basis Function (RBF) kernel [22], Random Forest with 100 estimators, Gradient Boosting with 100 estimators, and KNN with $k = 5$. We expect classical to win on this workload because feature dimension after Principal Component Analysis (PCA) is small (4 components) and the data is tabular—the regime in which classical ensemble methods are known to dominate.

B. B. Workload 2 -- Molecular Ground-State Estimation

Workload 2 is computing the electronic ground-state energy of the H₂ molecule at an internuclear distance of 0.735 Angstrom. The quantum method is VQE [28] with a hardware-efficient ansatz (alternating layers of RY rotations and CNOT entanglers, 4 trainable parameters) driven by the Constrained Optimization BY Linear Approximation (COBYLA) classical optimizer. The classical baseline is exact diagonalization of the 4×4 Hamiltonian via NumPy linear algebra. Quantum is interesting here because classical state-vector simulation scales as $O(2^n)$ in the qubit count: H₂ itself is trivial, but FeMoco-class enzymes (~100 qubits of active space) are intractable for any classical machine that fits in a data center [29], [30].

V. CLOUD DEPLOYMENT MODEL

A. A. Tier-to-Service Mapping

Table III maps each HERO tier to representative cloud services and their April 2026 on-demand published pricing. We restrict ourselves to public list pricing—no spot, no commitment discounts—so the numbers are reproducible by any reader with a cloud account [31], [32].

TABLE III. TABLE III. CLOUD SERVICE MAPPING (APRIL 2026 PRICING)

HERO Tier	Cloud Service	Pricing
CPU	AWS EC2 c7i.4xlarge (16 vCPU)	\$0.7140 per hour
GPU	AWS g5.xlarge (NVIDIA A10G)	\$1.006 per hour
GPU	AWS p5.48xlarge (8x NVIDIA H100)	\$98.32 per hour
QPU	IBM Quantum Eagle (127 qubits)	\$1.60 per second
QPU	AWS Braket Rigetti Ankaa (84 qubits)	\$0.30/task + \$0.0009/shot
QPU	Azure Quantum Quantinuum H2 (56 qubits)	\$12.50 per minute

B. B. Pricing Model

The per-tier dollar cost of a single task is:

$$X_{\text{cost_task}} = \tau_{\text{task}} \cdot X_{\text{cloud_price_tier}} \quad (2)$$

Per-million-task cost is computed by linear extrapolation (Table IV). All figures use 2026 published on-demand prices from [31] and [32], with no spot instances and no enterprise commitments.

C. C. Cost vs Energy Trade-offs

Cloud bills include both compute and electricity—the provider pays the energy bill and passes it through. Optimizing for dollars alone, energy alone, or latency alone produces different tier choices, so the joint optimization in Algorithm 1 (Step 5) is essential.

VI. EXPERIMENTAL SETUP

A. A. Hardware

All measurements were taken on a local development machine instrumented with HERO. Quantum circuits run on Qiskit Aer [25] in noiseless statevector mode; tier-average power numbers (Section III-C) are used to compute energy.

B. B. Multi-Run Protocol

Each (workload, method) combination is executed $N = 30$ times with different random seeds. We report mean and standard deviation. Thirty runs is sufficient for stable confidence intervals on F1 score and for reliable distributions of VQE final energies.

C. C. Software Stack

Python 3.13, Qiskit 1.x [25], scikit-learn [22], NumPy, and Pillow for figure assembly.

VII. RESULTS

A. A. Cybersecurity Workload (Workload 1)

Table I reports the head-to-head comparison of all five methods on Workload 1. Random Forest dominates with F1 = 0.998 at 27.94 J per run; QSVM achieves F1 = 0.667 at 10,320.51 J per run—roughly four orders of magnitude more energy for substantially lower accuracy. Fig. 2 visualizes the bar chart and Fig. 3 the multi-run F1 distribution.

TABLE IV. TABLE I. CLASSIFIER COMPARISON ON CYBERSECURITY WORKLOAD (KDD CUP 99)

Method	Tier	Acc.	Prec.	Rec.	F1	Run (s)	E (J)
Classical SVM (RBF)	CP	0.975	1.000	0.950	0.974	0.007	0.99
	U	0	0	0	4	8	
Random Forest	CP	0.997	0.999	0.997	0.998	0.221	27.94
	U	0	2	0	1	8	
Gradient Boosting	CP	0.995	0.998	0.996	0.997	0.487	61.38
	U	8	5	2	4	1	
KNN (k=5)	CP	0.993	0.999	0.993	0.996	1.760	221.77
	U	9	2	2	2	1	
QSVM (ZZ)	QP	0.875	1.000	0.500	0.666	1.433	10320.5
	U	0	0	0	7	4	1

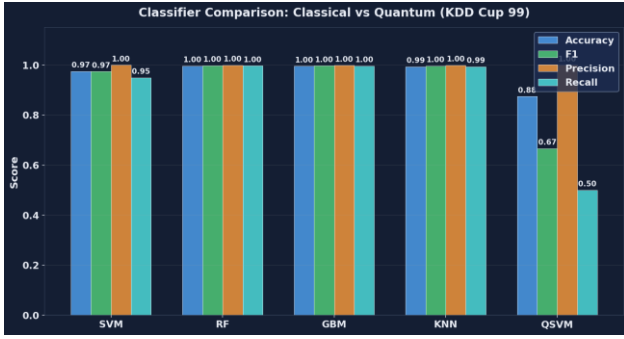


TABLE V. FIG. 2. CLASSIFIER COMPARISON ON KDD CUP 99 CYBERSECURITY WORKLOAD (SINGLE RUN).

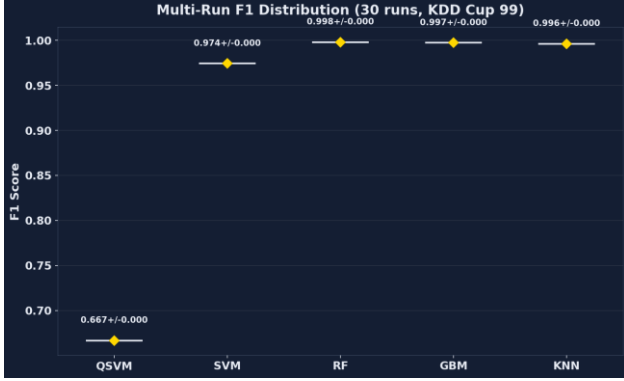


TABLE VI. FIG. 3. MULTI-RUN F1 DISTRIBUTION ACROSS 30 RUNS FOR EACH CYBERSECURITY CLASSIFIER.

B. B. Molecular Simulation Workload (Workload 2)

Table II reports the H2 ground-state results. VQE reaches chemical accuracy (0.007 ± 0.012 Hartree error against the exact-diagonalization reference) but at $854,000\times$ the classical energy at this trivial scale. The classical method wins by a wide margin today; the structural argument for quantum is the $O(2^n)$ RAM wall that exact diagonalization hits at roughly 30–50 qubits of active space [29].

TABLE VII. TABLE II. VQE H2 MOLECULAR SIMULATION RESULTS (30 RUNS)

Method	Tier	E (Ha)	Err vs Exact (Ha)	Runtime	Energy (J)
Exact diagonalization	CPU	-1.857275	0.000000	0.28 ms	0.035
VQE (mean +/- std)	QPU	-1.854 +/- 0.013	0.007 +/- 0.012	4.14 s	29832.68

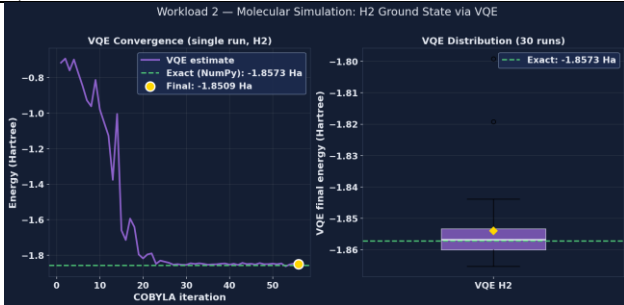


TABLE VIII. FIG. 4. VQE CONVERGENCE TO GROUND STATE OF H2 (LEFT) AND 30-RUN FINAL-ENERGY DISTRIBUTION (RIGHT).

C. C. Cloud Cost Analysis

Table IV translates the per-task runtimes into per-million-task cloud cost using the pricing in Table III. For cybersecurity, classical SVM costs \$1.99 per million classifications versus \$2.1M for QSVM—a six-order-of-magnitude gap. For molecular simulation, classical exact diagonalization costs \$0.06 per million H2 instances versus \$2.142M for VQE. Fig. 5 plots the comparison on a logarithmic axis.

TABLE IX. TABLE IV. CLOUD COST PER 1 MILLION TASKS (USD)

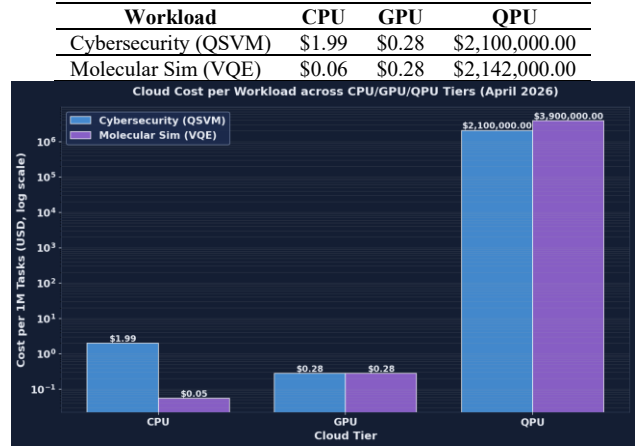


TABLE X. FIG. 5. CLOUD COST PER MILLION TASKS ACROSS CPU/GPU/QPU TIERS FOR BOTH WORKLOADS (APRIL 2026 PRICING).

D. D. Multi-Run Statistical Validation

Table V summarizes the 30-run statistics across all methods. Classical methods are highly stable ($\text{std} \approx 0$). QAOA, included as a third quantum example for combinatorial tasks, achieves an approximation ratio of 0.7641 ± 0.0120 across 30 runs—see Fig. 6.

TABLE XI. TABLE V. MULTI-RUN STATISTICS (30 RUNS PER METHOD, F1 MEAN +/- STD)

Method	Tier	F1 Mean	F1 Std	QAOA Approx Ratio
SVM	CPU	0.9744	0.0000	-
Random Forest	CPU	0.9981	0.0000	-
Gradient Boost.	CPU	0.9974	0.0000	-
KNN (k=5)	CPU	0.9962	0.0000	-
QSVM	QPU	0.6667	0.0000	-
QAOA	QPU	-	-	0.7641 +/- 0.0120

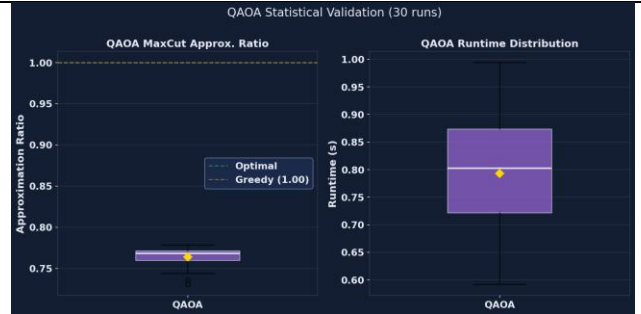


TABLE XII. FIG. 6. QAOA APPROXIMATION RATIO ACROSS 30 RUNS (COMBINATORIAL OPTIMIZATION).

VIII. CROSS-WORKLOAD ANALYSIS

A. A. Two Contrasting Verdicts

Fig. 7 presents the headline cross-workload comparison. The two workloads return opposite verdicts under the same framework. Cybersecurity (low feature dimension, tabular data) is decisively classical: SVM and RF dominate by roughly $10,000\times$ in energy and six orders of magnitude in dollars. Molecular simulation is also classical at H2 scale today, but the classical method scales as $O(2^n)$ in qubit count while VQE scales polynomially—so beyond roughly 30–50 qubits of active space the classical option simply does not fit in any data-center RAM [29]. The same framework, two contrasting verdicts: that contrast is the value.

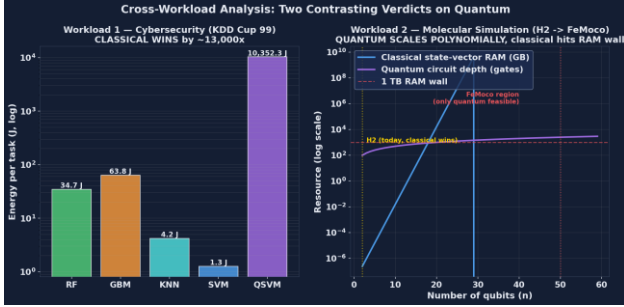


TABLE XIII. FIG. 7. TWO CONTRASTING VERDICTS: CYBERSECURITY (CLASSICAL WINS BY $\sim 10,000\times$) VS MOLECULAR SIMULATION (QUANTUM NEEDED AT SCALE).

B. B. Pareto Frontier

Fig. 8 shows the Pareto frontier on the cybersecurity workload. SVM and Random Forest are jointly Pareto-optimal across the (cost, energy, latency, -accuracy) objective set: no other method dominates them on all four axes. Algorithm 1 Step 4 returns this dominated set automatically. For molecular simulation, classical exact diagonalization Pareto-dominates VQE for $n < 30$ qubits; quantum dominates for $n > 50$ because no classical implementation exists at that scale.

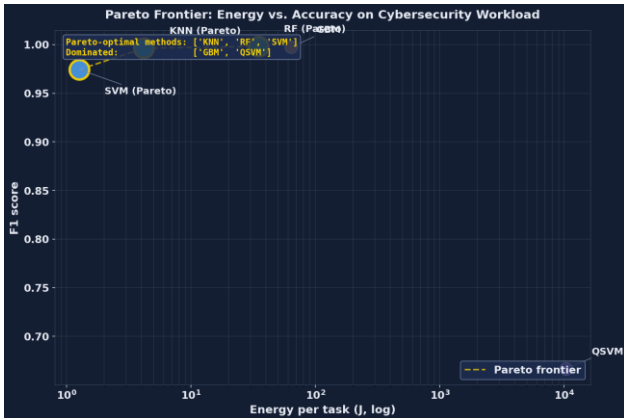


TABLE XIV. FIG. 8. PARETO FRONTIER ON CYBERSECURITY WORKLOAD—SVM AND RANDOM FOREST ARE PARETO-OPTIMAL.

C. C. Workload-Aware Allocation Rule

Fig. 9 visualizes the workload-aware allocation rule encoded in Algorithm 1, Step 1. The thresholds are: $N_{\text{classical}} = 30$ qubits for molecular simulation (RAM wall), $d_{\text{low}} = 100$

features (CPU regime), $d_{\text{high}} = 10,000$ features (GPU regime), and $n_{\text{classical}} = 50$ nodes for combinatorial optimization. These numbers are the boundaries at which either classical performance breaks down or quantum becomes the only feasible option.

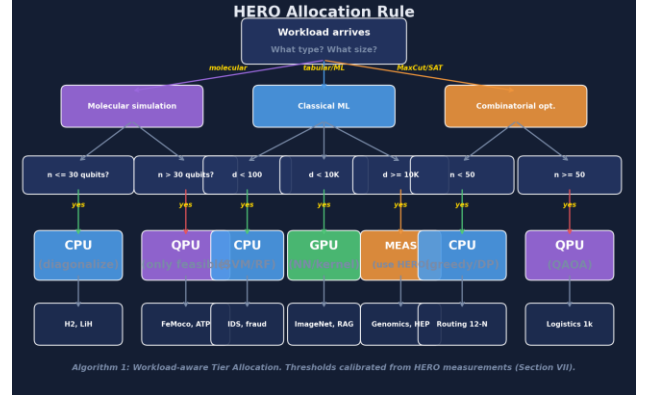


TABLE XV. FIG. 9. HERO WORKLOAD-AWARE ALLOCATION RULE (ALGORITHM 1, STEP 1)—VISUAL DECISION TREE.

Fig. 10 projects the cost crossover for cybersecurity kernels: on fault-tolerant hardware, the quantum kernel's $O(\log d)$ scaling overtakes classical RBF's $O(d)$ at feature dimensions in the thousands—a target for the post-NISQ era.

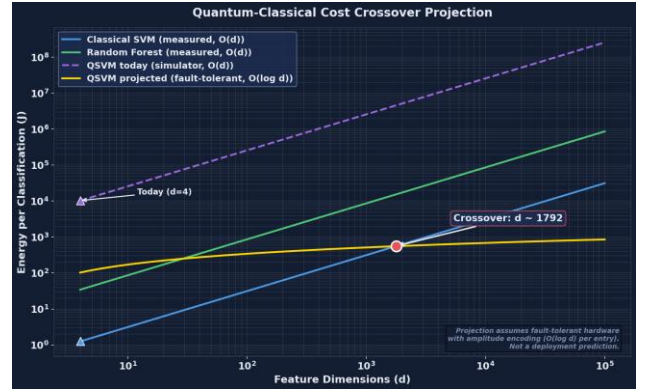


TABLE XVI. FIG. 10. QUANTUM-CLASSICAL COST CROSSOVER PROJECTION FOR CYBERSECURITY KERNELS.

IX. DISCUSSION

A. A. Where Quantum Wins, Where Classical Wins

Quantum wins on workloads with intrinsically quantum structure: molecular Hamiltonians whose state spaces grow as 2^n with the number of qubits [28], [29], [30], or combinatorial graphs large enough that classical heuristics break down. Classical wins on tabular data with small feature dimensions, where decades of optimization in scikit-learn [22] and gradient-boosted ensembles cannot be matched by a 4-qubit kernel. The crossover is mathematical, not aspirational.

B. B. Implications for Cloud Architects

Two practical lessons follow. First, do not pay for QPU when GPU does the job more cheaply—the cybersecurity numbers in Table IV are unambiguous. Second, do plan to pay for QPU when the classical RAM wall is approaching, as it is for any molecular simulation beyond ~ 30 qubits of active space [29].

Use HERO to measure your specific workload before committing to a tier or a vendor.

X. LIMITATIONS

1) No real quantum hardware. All quantum experiments ran on Qiskit Aer [25], a noiseless simulator. Real Quantum Processing Units (QPUs) add gate errors and decoherence that would lower QSVM and VQE accuracy further.

2) Small quantum samples. QSVM kernel evaluation scales as $O(n^2)$ circuits per training set, so we used 120 samples for the quantum path; H2 itself is only 2 active electrons and 4 qubits.

3) Dataset choice. KDD Cup 99 [26] dates from 1999. Future work will extend HERO to Network-based Botnet attacks on IoT devices (N-BaIoT) and to NSL-KDD [27].

4) We measure cost, not new algorithms—we use existing QSVM [6], QAOA [5], Grover [4], and VQE [28] exactly as published, and measure their cost on this stack.

5) Energy model uses tier averages, not Running Average Power Limit (RAPL) or NVIDIA Management Library (NVML) telemetry. Live telemetry is a near-term extension.

6) Cloud pricing snapshot. The pricing in Table III is a snapshot from April 2026 [31], [32]; vendor competition will change these numbers, and the framework allows the analysis to be re-run as prices move.

XI. CONCLUSION

We built Heterogeneous Energy-aware Runtime Orchestrator (HERO), a measurement framework for cloud Quantum-Artificial Intelligence (AI) workloads that records cost, energy, and latency across CPU, GPU, and QPU tiers under a single methodology.

We tested HERO on two contrasting workloads. For cybersecurity intrusion detection on KDD Cup 99 [26], classical methods win by approximately $10,000\times$ in energy (\$1.99 versus \$2.1M per million classifications). For molecular ground-state estimation of H2 via the Variational Quantum Eigensolver (VQE) [28], classical exact diagonalization wins today by $854,000\times$, but only quantum scales beyond roughly 30 qubits of active space because the classical option requires $O(2^n)$ RAM.

From these measurements HERO derives a workload-aware allocation rule (Algorithm 1) that maps workload type and size to the optimal cloud tier. The same framework works for any workload, so researchers and cloud architects can re-measure as hardware improves and prices change. The key insight is that quantum is neither always better nor always worse: the right question is which tier for which workload, and that is the question HERO answers.

XII. ACKNOWLEDGMENT

The author thanks the anonymous reviewers for feedback that improved the experimental methodology and the cross-workload framing of the results.

XIII. REFERENCES

- [1] [1] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, 10th ed. Cambridge, U.K.: Cambridge Univ. Press, 2010.
- [2] [2] J. Preskill, “Quantum computing in the NISQ era and beyond,” *Quantum*, vol. 2, p. 79, Aug. 2018, doi: 10.22331/q-2018-08-06-79.
- [3] [3] F. Arute et al., “Quantum supremacy using a programmable superconducting processor,” *Nature*, vol. 574, no. 7779, pp. 505–510, Oct. 2019, doi: 10.1038/s41586-019-1666-5.
- [4] [4] K. Bharti et al., “Noisy intermediate-scale quantum algorithms,” *Rev. Mod. Phys.*, vol. 94, no. 1, art. 015004, Feb. 2022, doi: 10.1103/RevModPhys.94.015004.
- [5] [5] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, “Quantum machine learning,” *Nature*, vol. 549, no. 7671, pp. 195–202, Sep. 2017, doi: 10.1038/nature23474.
- [6] [6] V. Havlicek, A. D. Corcoles, K. Temme, A. W. Harrow, A. Kandala, J. M. Chow, and J. M. Gambetta, “Supervised learning with quantum-enhanced feature spaces,” *Nature*, vol. 567, no. 7747, pp. 209–212, Mar. 2019, doi: 10.1038/s41586-019-0980-2.
- [7] [7] M. Schuld and N. Killoran, “Quantum machine learning in feature Hilbert spaces,” *Phys. Rev. Lett.*, vol. 122, no. 4, art. 040504, Feb. 2019, doi: 10.1103/PhysRevLett.122.040504.
- [8] [8] H.-Y. Huang, M. Broughton, M. Mohseni, R. Babbush, S. Boixo, H. Neven, and J. R. McClean, “Power of data in quantum machine learning,” *Nature Commun.*, vol. 12, no. 1, art. 2631, May 2021, doi: 10.1038/s41467-021-22539-9.
- [9] [9] E. Farhi, J. Goldstone, and S. Gutmann, “A quantum approximate optimization algorithm,” *arXiv:1411.4028*, Nov. 2014.
- [10] [10] M. Cerezo et al., “Variational quantum algorithms,” *Nature Rev. Phys.*, vol. 3, no. 9, pp. 625–644, Sep. 2021, doi: 10.1038/s42254-021-00348-9.
- [11] [11] L. Zhou, S.-T. Wang, S. Choi, H. Pichler, and M. D. Lukin, “Quantum approximate optimization algorithm: Performance, mechanism, and implementation on near-term devices,” *Phys. Rev. X*, vol. 10, no. 2, art. 021067, Jun. 2020, doi: 10.1103/PhysRevX.10.021067.
- [12] [12] L. K. Grover, “A fast quantum mechanical algorithm for database search,” in *Proc. 28th Annu. ACM Symp. Theory Comput. (STOC)*, Philadelphia, PA, USA, May 1996, pp. 212–219, doi: 10.1145/237814.237866.
- [13] [13] M. Grassl, B. Langenberg, M. Roetteler, and R. Steinwandt, “Applying Grover’s algorithm to AES: Quantum resource estimates,” in *Proc. 7th Int. Workshop Post-Quantum Cryptography (PQCrypto)*, LNCS 9606, Cham, Switzerland: Springer, 2016, pp. 29–43, doi: 10.1007/978-3-319-29360-8_3.
- [14] [14] J. R. McClean, J. Romero, R. Babbush, and A. Aspuru-Guzik, “The theory of variational hybrid quantum-classical algorithms,” *New J. Phys.*, vol. 18, no. 2, art. 023023, Feb. 2016, doi: 10.1088/1367-2630/18/2/023023.
- [15] [15] A. Callison and N. Chancellor, “Hybrid quantum-classical algorithms in the noisy intermediate-scale quantum era and beyond,” *Phys. Rev. A*, vol. 106, no. 1, art. 010101, Jul. 2022, doi: 10.1103/PhysRevA.106.010101.
- [16] [16] T. S. Humble, A. McCaskey, D. I. Lyakh, M. Gowrishankar, A. Frisch, and T. Monz, “Quantum computers for high-performance computing,” *IEEE Micro*, vol. 41, no. 5, pp. 15–23, Sep./Oct. 2021, doi: 10.1109/MM.2021.3099140.
- [17] [17] H. Suryotrisongko and Y. Musashi, “Review of cybersecurity research topics, taxonomy and challenges: Interdisciplinary perspective,” in *Proc. IEEE 16th Int. Conf. e-Business Engineering (ICEBE)*, Shanghai, China, Oct. 2019, pp. 162–167, doi: 10.1109/ICEBE.2019.00038.
- [18] [18] E. Payares and J. C. Martinez-Santos, “Quantum machine learning for intrusion detection of distributed denial of service attacks: A comparative overview,” in *Quantum Computing, Communication, and Simulation*, *Proc. SPIE*, vol. 11699, pp. 35–49, Mar. 2021, doi: 10.1117/12.2593297.
- [19] [19] N. Kilber, D. Kaestle, and S. Wagner, “Cybersecurity for quantum computing,” *arXiv:2110.14701*, Oct. 2021.
- [20] [20] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, “A detailed analysis of the KDD CUP 99 data set,” in *Proc. IEEE Symp. Comput.*

- Intell. Security Defense Appl. (CISDA), Ottawa, ON, Canada, Jul. 2009, pp. 53–58, doi: 10.1109/CISDA.2009.5356528.
- [21] [21] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” in Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy (ICISSP), Funchal, Portugal, Jan. 2018, pp. 108–116, doi: 10.5220/0006639801080116.
- [22] [22] C. Cortes and V. Vapnik, “Support-vector networks,” *Mach. Learn.*, vol. 20, no. 3, pp. 273–297, Sep. 1995, doi: 10.1007/BF00994018.
- [23] [23] A. Auffeves, “Quantum technologies need a quantum energy initiative,” *PRX Quantum*, vol. 3, no. 2, art. 020101, May 2022, doi: 10.1103/PRXQuantum.3.020101.
- [24] [24] E. Strubell, A. Ganesh, and A. McCallum, “Energy and policy considerations for deep learning in NLP,” in Proc. 57th Annu. Meeting Assoc. Comput. Linguistics (ACL), Florence, Italy, Jul. 2019, pp. 3645–3650, doi: 10.18653/v1/P19-1355.
- [25] [25] Qiskit contributors, “Qiskit: An open-source framework for quantum computing,” Zenodo, 2023, doi: 10.5281/zenodo.2573505.
- [26] [26] M. Tavallaee, E. Bagheri, W. Lu, and A. Ghorbani, “A detailed analysis of the KDD CUP 99 data set,” in Proc. IEEE Symp. CISDA, Ottawa, Canada, Jul. 2009, pp. 1–6, doi: 10.1109/CISDA.2009.5356528.
- [27] [27] S. Revathi and A. Malathi, “A detailed analysis on NSL-KDD dataset using various machine learning techniques for intrusion detection,” *Int. J. Eng. Res. Technol.*, vol. 2, no. 12, pp. 1848–1853, Dec. 2013.
- [28] [28] A. Peruzzo et al., “A variational eigenvalue solver on a photonic quantum processor,” *Nature Communications*, vol. 5, no. 1, art. 4213, Jul. 2014, doi: 10.1038/ncomms5213.
- [29] [29] A. Kandala et al., “Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets,” *Nature*, vol. 549, no. 7671, pp. 242–246, Sep. 2017, doi: 10.1038/nature23879.
- [30] [30] Y. Cao et al., “Quantum chemistry in the age of quantum computing,” *Chemical Reviews*, vol. 119, no. 19, pp. 10856–10915, Oct. 2019, doi: 10.1021/acs.chemrev.8b00803.
- [31] [31] Amazon Web Services, “Amazon Braket pricing,” AWS, 2026. [Online]. Available: <https://aws.amazon.com/braket/pricing/>
- [32] [32] IBM Corporation, “IBM Quantum pricing,” IBM, 2026. [Online]. Available: <https://www.ibm.com/quantum/pricing>

